

Clase 3: La Comunicación entre Usuario y Servidor

En esta sesión daremos el paso más importante hacia el desarrollo de aplicaciones reales: aprenderemos a escuchar al usuario. Pasaremos de crear sitios que solo "muestran" información a construir sistemas que "reciben, procesan y validan" datos.

Objetivos de la sesión

- Comprender la diferencia técnica y de seguridad entre **GET** y **POST**.
- Aprender a recibir datos desde un formulario HTML en Python.
- Implementar validaciones de seguridad y lógica de negocio (Email, Teléfono).
- Mostrar retroalimentación visual al usuario (Alertas).

Momentos de la Clase

1. Fundamentos: ¿Cómo viaja la información?

Para que un navegador (Cliente) le envíe datos a Python (Servidor), debe utilizar métodos de transporte. Los dos más importantes son:

A. El Método GET (La Postal)

Imagina que envías una postal por correo. El cartero y cualquier persona que la vea puede leer el mensaje.

- **Comportamiento:** Los datos se escriben directamente en la dirección URL (ej. `google.com/search?q=python`).
- **Uso:** Se utiliza para búsquedas, filtros o navegación simple. Nunca para datos privados.

B. El Método POST (El Sobre Sellado)

Imagina que envías una carta dentro de un sobre lacrado. El contenido está protegido y no se ve desde el exterior.

- **Comportamiento:** Los datos viajan ocultos en el cuerpo de la petición.
- **Uso:** Es el estándar de seguridad. Se usa obligatoriamente para registros, inicios de sesión y envío de datos sensibles.

2. El "Oído" del Servidor: El objeto request

Flask tiene un asistente especial llamado request. Su trabajo es capturar la "mochila" de datos que envía el usuario y entregársela a Python de forma organizada. Cuando el usuario presiona "Enviar", nosotros usamos este objeto para extraer cada campo por el nombre que le asignamos en el diseño.

3. Guía de Montaje: Paso a Paso

Para que tu sistema de contactos funcione correctamente, debes seguir este orden lógico, para ello utilizamos el mismo proyecto de hace una semana:

```
proyecto_clase2/
├── app.py          # El cerebro (Python)
├── templates/      # Carpeta obligatoria (en minúsculas y plural)
│   ├── base.html  # El molde general
│   ├── index.html # La página de inicio
│   └── servicios.html # La lista de servicios
└── contacto.html  # Formulario de contacto
```

Paso 1: Habilitar la recepción en Python

Por seguridad, Flask bloquea la recepción de datos a menos que tú le des permiso explícito. En tu archivo de Python, debes modificar la ruta para que acepte tanto visitas normales (GET) como envíos de datos (POST).

Colocaremos la librería **request** justo en la sección librerías, como se muestra a continuación:

```
from flask import request # Importamos request para manejar datos del formulario
```

También colocaremos una nueva ruta de la siguiente forma:

```
# --- RUTA DE CONTACTO ---

# Importante: Debemos permitir explícitamente el método POST para recibir datos.
@app.route('/contacto', methods=['GET', 'POST'])
def contacto():
    # Variables para controlar los mensajes en la interfaz
    mensaje_error = None
    mensaje_exito = None

    # Verificamos si el usuario envió el formulario (POST)
    if request.method == 'POST':
        # 1. RECOLECCIÓN DE DATOS
        # Usamos .get() para evitar errores si el campo no existe
        nombre = request.form.get('nombre')
        email = request.form.get('email')
        telefono = request.form.get('telefono')
        mensaje = request.form.get('mensaje')

        # 2. VALIDACIONES DE SEGURIDAD (Lógica de Negocio)

        # Validación: Campos vacíos
        if not nombre or not email or not telefono:
            mensaje_error = "⚠️ Los campos Nombre, Email y Teléfono son obligatorios."

        # Validación: Formato de Email básico
        elif "@" not in email or "." not in email:
            mensaje_error = "✉️ El formato del correo electrónico no es válido."

        # Validación: Teléfono (solo números y longitud mínima)
        elif not telefono.isdigit() or len(telefono) < 7:
            mensaje_error = "☎️ Por favor, ingresa un número de teléfono válido (solo números)."
```

```
# 3. PROCESAMIENTO EXITOSO
else:
    # Aquí es donde más adelante guardaremos en Base de Datos
    mensaje_exito = f"✅ ¡Perfecto {nombre}! Hemos recibido tu información."
    print(f"DEBUG: Datos recibidos -> {nombre}, {email}, {telefono}")

# Renderizamos la plantilla pasando los mensajes (si existen)
return render_template('contacto.html',
                       titulo="Registro de Contacto",
                       error=mensaje_error,
                       exito=mensaje_exito)
```

Paso 2: Configurar el "Mensajero" (HTML)

El formulario en HTML es el encargado de llevar los datos al servidor. Para que no se pierda, requiere tres elementos:

1. **Action:** La dirección exacta a donde debe ir (la ruta de tu función en Python).
2. **Method:** Especificar que el envío es tipo POST.
3. **Name:** El nombre interno de cada casilla. **Sin el atributo name, Python no podrá reconocer qué dato pertenece a qué campo.**

En el index.html colocar de forma anidada el siguiente enrutamiento:

```
<a href="/contacto" class="inline-block mt-8 ml-4 bg-green-600 text-white px-6 py-3
rounded-lg hover:bg-green-700">
    Contáctanos
</a>
```

En el archivo base.html ajustaremos el nuevo enlace de contacto sobre el encabezado principal:

```
<a href="/contacto" class="hover:text-indigo-200">Contacto</a>
```

En el archivo contacto.html haremos la siguiente configuración:

```
{% extends "base.html" %}

{% block content %}
<div class="max-w-2xl mx-auto mt-10">
  <div class="bg-white shadow-2xl rounded-3xl overflow-hidden">

    <!-- Encabezado del Formulario -->
    <div class="bg-indigo-700 p-6 text-white text-center">
      <h2 class="text-3xl font-bold">Contáctanos</h2>
      <p class="text-indigo-100 mt-2">Completa el formulario para enviarnos tus datos.</p>
    </div>

    <div class="p-8">

      <!-- ALERTA DE ERROR -->
      {% if error %}
      <div class="bg-red-50 border-l-4 border-red-500 p-4 mb-6 animate-pulse">
        <div class="flex">
          <div class="flex-shrink-0 text-red-500 text-xl">⚠️</div>
          <div class="ml-3">
            <p class="text-sm text-red-700 font-medium">{{ error }}</p>
          </div>
        </div>
      </div>
      {% endif %}

      <!-- ALERTA DE ÉXITO -->
      {% if exito %}
      <div class="bg-green-50 border-l-4 border-green-500 p-4 mb-6">
        <div class="flex">
          <div class="flex-shrink-0 text-green-500 text-xl">✅</div>
          <div class="ml-3">
            <p class="text-sm text-green-700 font-bold">{{ exito }}</p>
          </div>
        </div>
      </div>
      {% endif %}
    </div>
  </div>
</div>
```

```

        </div>
    </div>
</div>
{% endif %}

<!-- FORMULARIO -->
<form action="/contacto" method="POST" class="space-y-6">

    <div>
        <label class="block text-sm font-semibold text-gray-700 mb-1">Nombre
        Completo</label>
        <input type="text" name="nombre" placeholder="Ej: Juan Pérez"
            class="w-full px-4 py-3 border border-gray-300 rounded-xl focus:ring-2
            focus:ring-indigo-500 focus:border-transparent outline-none transition shadow-sm">
    </div>

    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div>
            <label class="block text-sm font-semibold text-gray-700 mb-1">Correo
            Electrónico</label>
            <input type="text" name="email" placeholder="correo@ejemplo.com"
                class="w-full px-4 py-3 border border-gray-300 rounded-xl focus:ring-2
                focus:ring-indigo-500 outline-none transition shadow-sm">
        </div>
        <div>
            <label class="block text-sm font-semibold text-gray-700 mb-1">Teléfono</label>
            <input type="text" name="telefono" placeholder="300 123 4567"
                class="w-full px-4 py-3 border border-gray-300 rounded-xl focus:ring-2
                focus:ring-indigo-500 outline-none transition shadow-sm">
        </div>
    </div>

    <div>
        <label class="block text-sm font-semibold text-gray-700 mb-1">Mensaje
        (Opcional)</label>
        <textarea name="mensaje" rows="3"
            class="w-full px-4 py-3 border border-gray-300 rounded-xl focus:ring-2
            focus:ring-indigo-500 outline-none transition shadow-sm"></textarea>
    </div>

```

```

        </div>

        <button type="submit"
            class="w-full bg-indigo-600 hover:bg-indigo-800 text-white font-bold py-4
            rounded-xl shadow-lg transform active:scale-95 transition-all">
            Enviar Registro Seguro
        </button>
    </form>

</div>
</div>
</div>
{% endblock %}

```

Paso 3: El Filtro de Seguridad (Validación)

Un desarrollador profesional nunca confía ciegamente en lo que el usuario escribe. El servidor debe actuar como un filtro de calidad que verifique:

- Que no haya campos vacíos.
- Que el correo electrónico tenga una estructura real (como el símbolo @).
- Que el teléfono sea un número válido y no texto al azar.

Paso 4: Respuesta al Usuario (Feedback)

Si el usuario comete un error, el sistema debe avisarle de inmediato con un mensaje claro. Si el envío es exitoso, debe confirmarlo. Usamos "Alertas Estilizadas" que aparecen solo cuando Python detecta que algo sucedió.

Errores Críticos que debemos evitar

- **Confundir ID con NAME:** En CSS usamos el id para el diseño, pero para que Python reciba el dato **necesitas el atributo name**.
- **Olvidar importar request:** Si no importas la herramienta al inicio de tu código de Python, el servidor no podrá "leer" el formulario.
- **No usar HTTPS en producción:** Aunque en clase trabajamos de forma local, en el mundo real todos los formularios deben viajar por conexiones seguras para proteger la privacidad.



Tips

- **El atributo name:** Recuérdales mil veces que el name en el HTML debe ser EXACTAMENTE igual al que llaman en Python. Es el error más común.
- **Métodos en la ruta:** No olvides explicar que ahora las rutas necesitan permiso explícito: `methods=['GET', 'POST']`.



Extra

La mejora que necesita el archivo `servicios.html` es la siguiente (en caso de que no lo hubieran realizado)

```
{% extends "base.html" %}

{% block content %}
<div class="max-w-6xl mx-auto mt-10 space-y-6">
  <h2 class="text-4xl font-bold text-indigo-700 mb-6">{{ titulo }}</h2>

  <div class="grid gap-6 md:grid-cols-2 lg:grid-cols-3">
    {% for s in servicios %}
      <div class="bg-white rounded-3xl shadow-lg border border-gray-200 p-6">
        <h3 class="text-2xl font-semibold text-gray-800">{{ s.nombre }}</h3>
        <p class="text-gray-500 mt-2">Precio: <span class="font-semibold">${{ s.precio }}</span></p>
        <div class="mt-4">
          {% if s.disponible %}
            <span class="inline-flex items-center px-3 py-1 rounded-full bg-green-100 text-green-700">✅ Disponible</span>
          {% else %}
            <span class="inline-flex items-center px-3 py-1 rounded-full bg-red-100 text-red-700">❌ No disponible</span>
          {% endif %}
        </div>
      </div>
    {% endfor %}
  </div>
</div>
```


{% endfor %}

</div>

</div>

{% endblock %}